



Code of Conduct:

I AKSHAYA SEKAR/1911260204 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Akshaya Sekar



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer: Testing should be treated as a planned engineering activity rather than random execution. First, the team should study requirements, identify risks, define test objectives, and prepare a test plan. Suitable techniques such as equivalence partitioning, boundary value analysis, decision-table testing, and risk-based testing should be selected. Test cases must contain clear inputs, preconditions, steps, and expected results. A traceability relationship between requirements and test cases helps ensure adequate coverage. For example, instead of randomly testing a login page, the team can systematically test valid credentials, invalid credentials, blank fields, locked accounts, and boundary conditions. Test results and defects should be recorded and reviewed. This structured approach makes testing repeatable, measurable, and more effective, thereby improving quality and defect detection.

2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in real-time systems. Explain the benefits with relevant examples.

Answer: A well-defined testing process gives the banking project a consistent method for planning, designing, executing, and monitoring tests. It should include requirement review, test planning, test-case design, unit and integration testing, system testing, regression testing,



defect reporting, and test closure. For a banking system, critical functions such as login, balance enquiry, fund transfer, withdrawal, and transaction reversal should be tested systematically. Risk-based testing can give higher priority to financial transactions and security-related functions. Every defect should be documented with severity, priority, steps to reproduce, and status. Regression tests should be repeated after fixes and new releases. Such a process reduces missed defects, improves traceability, supports consistent results, and increases confidence that the banking system will operate reliably in real-time use.

3. During the development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects.

Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer: Testing should be performed throughout the SDLC instead of being postponed until the end. During requirements, testers review specifications for ambiguity and create acceptance criteria. During design, they review interfaces, data flow, and risks. During coding, unit testing verifies individual components. Integration testing checks communication between modules, while system testing validates the complete e-commerce application. Acceptance testing confirms that business requirements are satisfied. Regression testing is performed whenever changes are introduced. For example, an order-processing system should separately test product selection, payment, inventory, shipping, and integration. Process discipline also requires documented test cases, defect tracking, entry and exit criteria, and review activities. This structured approach detects defects earlier, prevents their propagation between phases, improves traceability, and produces more predictable quality outcomes.

4. A login system shows a mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer: An error is a human mistake made by a developer, tester, analyst, or other project participants. A defect is the flaw introduced into a work product because of that error. A failure occurs when the software executes and produces an incorrect or unexpected result because of a defect. For example, suppose a developer writes a login condition incorrectly and allows an invalid password. The incorrect coding decision is the error, the incorrect login logic



in the program is the defect, and the system actually allowing the invalid user to log in is the failure. Testing primarily reveals failures and provides evidence that defects may exist. Debugging and analysis then locate and correct the underlying defect. Understanding these relationships helps the team report problems accurately and trace them to their causes.

5. A team struggles to detect defects early, leading to increased costs during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer: The principle of early testing should be applied to detect defects before they become expensive to correct. Testers should participate in requirements and design reviews and prepare test conditions while development is still in progress. Requirements can be checked for ambiguity, completeness, consistency, and testability. For example, if an online payment requirement does not clearly specify what should happen after a failed transaction, a tester can identify the ambiguity before coding begins. Developers can then clarify the expected behavior and create appropriate tests. Static reviews, unit testing, and continuous integration can further detect defects early. Early detection reduces rework because defects are corrected near their source rather than after integration or deployment. Therefore, applying the early-testing principle improves efficiency, reduces cost, and increases software quality.

6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer: A tester should contribute during the requirement phase by reviewing every requirement for clarity, completeness, consistency, feasibility, and testability. Ambiguous words such as “quickly,” “normally,” or “user-friendly” should be replaced with measurable acceptance criteria. In a healthcare system, testers should also check important data rules, validation conditions, privacy requirements, and error-handling expectations. They can prepare initial test conditions and trace each requirement to one or more tests. For example, if a patient-record requirement does not specify how duplicate records are handled, the tester can raise the issue before implementation. Early involvement prevents misunderstandings from being converted into design and coding defects. It also improves communication between



stakeholders, developers, and testers, reduces later rework, and helps ensure that the final system satisfies its intended healthcare requirements.

7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer: Defects in a payroll system can originate at different stages of the software lifecycle. An unclear requirement may be interpreted incorrectly during design, which can lead to incorrect algorithms and coding. For example, if the requirement does not clearly state how overtime is calculated, developers may implement a different rule from the organization's actual payroll policy. The resulting implementation becomes a defect and may cause incorrect salary output during execution. Requirement clarity can prevent this by using precise business rules, examples, data definitions, and measurable acceptance criteria. Requirements should be reviewed by business users, analysts, developers, and testers. Traceability should then connect requirements to design elements and test cases. Clear requirements reduce misunderstanding, improve testability, and prevent defects from propagating into later development stages.

8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer: The crash at boundary input values is a functional or input-validation defect associated with inadequate boundary-value coverage. Boundary Value Analysis is based on the observation that defects frequently occur at the limits of valid input ranges. If a field accepts values from 1 to 100, the tester should consider values such as 0, 1, 2, 99, 100, and 101, according to the requirement. The same principle can be applied to maximum length, minimum length, date ranges, file sizes, and numeric limits. If the application is tested only with normal middle-range values, boundary failures may remain hidden. By designing test cases around valid boundaries, just outside the boundaries, and relevant edge conditions, the team can expose incorrect validation logic and prevent crashes or unexpected behavior in production.



9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer: A defect repository stores historical information about defects and can be used as a source for improving future testing. Each record should include the defect description, affected module, severity, priority, environment, root cause, corrective action, and resolution. Testers can analyse repeated defects and identify modules or scenarios that require stronger coverage. For example, if previous releases repeatedly contain calculation defects in a billing module, new releases should include additional calculation, boundary, and regression tests for that module. Defect trends can also reveal weaknesses in requirements, design, coding, or testing practices. The repository should therefore be reviewed during test planning and regression-test selection. Using historical defect information prevents the team from repeating earlier mistakes, improves risk-based prioritization, increases regression coverage, and supports continuous improvement of software quality.

10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer: The ATM withdrawal problem is a functional defect because the system fails to perform a required business operation correctly. A complete functional test set should verify valid withdrawals, insufficient balance, invalid amounts, daily limits, account status, cancelled transactions, and correct balance updates. It should also verify that the ATM, bank server, and transaction database maintain consistent information. For example, when a valid withdrawal is made, cash should be dispensed, the account balance should be reduced by the correct amount, and the transaction should be recorded. If one of these results is incorrect, a functional defect exists. The defect could have been prevented through requirement-based test design, integration testing, negative testing, boundary testing, and regression testing after transaction logic changes. Critical financial workflows should receive high testing priority because failures can directly affect users and transactions.

11. A large application shows that most defects are concentrated in a few modules. The



testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer: Defect clustering means that a relatively large number of defects are often concentrated in a small number of modules or components. The testing team should analyze historical defect reports and identify modules with high defect density, frequent changes, complex logic, or repeated failures. These modules should receive greater testing attention and deeper test coverage. For example, in an e-commerce application, payment and order-processing modules may contain more defects than static information pages. Testers can therefore prioritize detailed functionality, integration, boundary, security, and regression tests for those high-risk areas. Less risky modules can receive appropriate but comparatively lower testing effort. Defect clustering helps allocate limited time and resources where they are most valuable. It improves testing efficiency, increases the probability of detecting important defects, and supports risk-based test planning.

12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer: The slow response of a web application under heavy user load is a performance defect, specifically related to load or capacity behavior. The team should classify and investigate performance testing. Load testing checks expected user volumes, while stress testing examines behavior beyond normal capacity. Important measures include response time, throughput, resource utilization, error rate, and system stability. For example, if a shopping website normally supports a specified number of concurrent users, but response time becomes unacceptable when that load is reached, the issue should be recorded as a performance defect. Classification helps the team assign the issue to the appropriate technical area and select suitable diagnostic methods. Developers can then investigate database queries, server resources, network delays, or inefficient application logic. Correct classification speeds up resolution and helps prevent similar performance failures in later releases.

13. A team is preparing for regression testing after multiple releases. They have access to historical data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer: Historical defect data can be reused to strengthen regression testing after multiple



releases. The team should review previous defects and identify the modules, functions, input conditions, and workflows that have failed before. Those scenarios should become permanent regression tests or receive higher priority during regression cycles. For example, if earlier releases contained defects in payment confirmation, the regression suite should always verify successful payment, failed payment, duplicate requests, and correct transaction status. Defect severity and recurrence can be used to prioritize the most important cases. The team can also analyze root causes and create additional tests around similar conditions. Reusing historical data prevents known failures from returning, improves coverage of high-risk areas, reduces the time spent on selecting tests, and makes regression testing more focused and effective.

14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer: The tester plays an important role in reducing integration defects by communicating requirements, interfaces, risks, and expected behavior clearly with developers. Testers should participate in requirement reviews, design discussions, interface planning, integration planning, and defect analysis instead of joining only after coding is complete. During integration testing, they should verify data flow, API or interface contracts, error handling, sequencing, and compatibility between modules. For example, if an order module sends an incorrect data format to the payment module, early interface review may identify the mismatch before full integration. Shared test conditions and regular developer-tester discussions improve mutual understanding. Early involvement also allows testers to identify ambiguities and risks before implementation. This reduces misunderstandings, minimizes integration defects, shortens debugging time, and supports smoother collaboration and better software quality.

15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer: Defect prevention focuses on removing the causes of recurring defects instead of repeatedly correcting the same symptoms. The team should analyze repeated defects using root-cause analysis and determine whether the source is unclear about requirements, design



weaknesses, coding practices, inadequate reviews, or missing tests. Peer reviews, requirement inspections, coding standards, checklists, automated static analysis, and improved test design can then be introduced. For example, if every release contains incorrect date calculations, the team should investigate the common cause and add clear date rules, review procedures, boundary tests, and reusable validation logic. Preventive actions should be monitored to confirm that the recurrence rate decreases. This approach improves the development process rather than only fixing individual failures. As a result, defect prevention reduces rework, maintenance effort, production failures, and long-term testing cost while improving overall software quality.

16. A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

Answer: The mobile application should use a standardized testing strategy so that different testers and modules follow consistent practices. The strategy should define test levels, test objectives, test-design techniques, environments, entry and exit criteria, defect-reporting rules, and regression procedures. Requirements should be traced to test cases, so important functions are not missed. Functional, usability, compatibility, installation, interruption, and performance tests should be included because mobile applications operate across devices and environments. For example, the team can maintain a common regression suite that is executed after every significant build. Defect reports should use a standard format containing steps, expected and actual results, severity, and environment details. Standardization makes testing repeatable and measurable, reduces variation between testers, improves coverage, and increases confidence that the application will behave reliably across supported mobile conditions.

17. A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer: Testing and debugging are related but different activities. Testing evaluates software to reveal failures and determine whether it meets specified requirements, while debugging investigates the cause of a failure and modifies the software to remove the underlying defect. For example, suppose a banking application displays an incorrect balance after a fund transfer. A tester executes the transaction, compares the actual balance with the expected balance, and



reports the failure with evidence. The developer then debugs the application by tracing the transaction calculation, identifying the incorrect logic, correcting it, and retesting the change. Testing does not normally determine the exact programming cause by itself, whereas debugging focuses on diagnosis and correction. After debugging, testing is repeated, and regression tests are executed to confirm that the fix works and has not introduced additional problems.

18. An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

Answer: A cross-browser testing strategy should first identify the browsers, versions, operating systems, and device combinations that the application officially supports. The team should create a common set of functional and visual test cases and execute them across the selected environments. Important scenarios include login, navigation, product search, cart operations, checkout, forms, payments, and error handling. Browser-specific rendering, JavaScript behavior, layout, responsiveness, and compatibility should also be checked. For example, a checkout function that works correctly in one browser but fails in another should be recorded with the browser version and environment details. Automated cross-browser tools can help execute repetitive regression tests, while manual checks can cover visual and usability issues. This systematic strategy identifies compatibility defects early, improves customer experience, and ensures consistent behavior across supported browsers.

19. A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer: Validation gaps in the logistics application may occur when requirements are incomplete; expected results are not clearly defined, or important input and workflow combinations are not tested. The tester should trace each requirement to test conditions and verify both normal and abnormal scenarios. Important cases include valid addresses, invalid locations, missing data, duplicate shipments, status changes, boundary values, and network or service failures. For example, if a shipment status can change from “Dispatched” to “Delivered,” the tester should verify valid transitions as well as invalid transitions that should be rejected. Actual outputs must be compared with clearly defined expected results. Proper



validation ensures that incorrect data and unexpected conditions are detected before release. It improves reliability, prevents inconsistent outputs, supports accurate logistics decisions, and increases confidence in the system's business behavior.

20. A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer: Testing as an engineering activity requires planned, systematic, and measurable test design rather than random execution. The team should begin by analysing requirements and identifying high-risk functions. Appropriate techniques such as equivalence partitioning, boundary value analysis, decision tables, state-transition testing, and exploratory testing can then be selected. Each test case should define preconditions, inputs, expected results, and traceability to a requirement. For example, a payment module should systematically test successful payment, invalid card details, insufficient funds, timeout, cancellation, and duplicate submission rather than selecting random inputs. Test results and defects should be documented and reviewed. A structured process improves coverage because important conditions are deliberately considered. It also makes testing repeatable, supports objective evaluation, reduces missed defects, and provides evidence that the software has been tested against its requirements.

21. A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer: The data inconsistency between healthcare modules may result from mismatched interface definitions, different data formats, incorrect field mappings, synchronization problems, or incomplete integration testing. Testers should first examine the interface contracts and trace the same data across all connected modules. They should verify field names, data types, formats, units, update timing, and error-handling rules. For example, a patient identifier entered in one module must remain consistent when retrieved by another module. Integration tests should cover valid data transfer, missing values, invalid values, duplicate records, update operations, and failure of recovery. Testers should also compare expected and actual data at each integration point. Proper integration testing detects inconsistencies before deployment and confirms that modules communicate correctly. This is



especially important in healthcare systems, where inaccurate or inconsistent information can affect subsequent processing and decisions.

22. A project experiences delays because testers are not involved early in development.

The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer: Testers should participate throughout the SDLC rather than being involved only after development is completed. During requirements, they review clarity, completeness, risks, and acceptance criteria. During design, they examine architecture, interfaces, data flow, and testability. During development, they prepare test cases and support unit and integration testing. During system testing, they verify end-to-end behavior, while acceptance testing confirms that business needs are met. Regression testing should continue after fixes and changes. For example, a tester can identify an ambiguous requirement before coding and prevent the developer from implementing the wrong behavior. Early collaboration improves communication between analysts, developers, and testers and allows defects to be detected closer to their source. It reduces rework and project delays, improves test planning, and increases the probability that the final product meets its intended requirements.

23. A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer: The failure during peak usage indicates inadequate test coverage of high-load and high-risk scenarios. The team should identify realistic peak user volumes, concurrent transactions, resource-intensive operations, and combinations of critical activities. Load, stress, endurance, and performance tests should be designed according to the system's expected operating conditions. For example, a ticket-booking or shopping application should be tested when many users search, add items, and complete transactions simultaneously. Coverage should also include error handling, database limits, network delays, and recovery from resource exhaustion. Requirements and risk analysis can be used to identify scenarios that must not be omitted. Better test coverage increases the chance of detecting failures before production, provides evidence about system capacity and stability, and helps the team correct bottlenecks before users experience peak-load failures.



24. A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer: A defect repository provides a central and organized method for recording and tracking software defects. Each defect should have a unique identifier and include a clear description, steps to reproduce, expected result, actual result, environment, severity, priority, assigned owner, status, and resolution information. Testers should update the record as the defect moves through investigation, correction, retesting, and closure. For example, a defect that causes incorrect invoice totals can be linked to the affected test case and later reused for regression testing. The repository also allows the team to identify recurring modules, defect trends, and common root causes. Proper documentation prevents information from being lost, improves communication between testers and developers, supports accurate status tracking, and provides historical evidence for future releases. Thus, defect management becomes more systematic and measurable.

25. A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer: The banking system should be redesigned around a structured, risk-based testing process. Requirements should first be reviewed for transaction rules, security expectations, validation conditions, and error handling. Test cases should then cover account login, balance enquiry, deposits, withdrawals, transfers, transaction limits, reversals, failed transactions, and concurrent operations. Boundary value analysis and negative testing should be used for transaction amounts and limits. Integration testing must verify communication between application, database, and external banking services. A strong regression suite should be executed after every change, with critical transaction tests given high priority. Defects should be documented and analyzed for root causes so repeated failures can be prevented. This structured approach improves traceability, coverage, and repeatability. It reduces transaction failures, detects defects earlier, and increases reliability and confidence in the banking system.

26. A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.



Answer: The testers should extend testing beyond normal inputs to include edge cases and boundary conditions. Boundary Value Analysis is particularly useful because defects often occur at the limits of permitted input ranges. If a system accepts an age from 18 to 60, relevant tests can include 17, 18, 19, 59, 60, and 61, along with other invalid or missing inputs where appropriate. The same idea applies to maximum field length, minimum quantity, dates, file size, and numerical limits. Equivalence partitioning can also divide inputs into valid and invalid groups so that representative cases are selected efficiently. Testing edge cases expose incorrect validation and unexpected behavior that normal-value testing may miss. Including these tests improves coverage, increases defect detection, and reduces the risk of failures occurring when users provide values close to system limits.

27. A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer: A process-based testing approach should define common activities and standards that every module follows. The organization can establish a standard test lifecycle covering requirement analysis, test planning, test design, environment preparation, execution, defect management, regression, and closure. Common templates should be used for test cases and defect reports, while entry and exit criteria should be defined for each test level. Metrics such as test execution status, defect density, severity distribution, and coverage can be monitored consistently. For example, every module can be required to demonstrate requirement traceability and completion of critical regression tests before release. Standardization does not mean every module receives identical tests; risk and functionality should still determine depth. A common process improves consistency, makes results comparable, reduces omissions, supports better communication, and produces more uniform quality across the application.

28. A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

Answer: The web application should use systematic input-validation testing to verify how it handles valid, invalid, unexpected, missing, and boundary inputs. Testers should identify the input rules for each field and create cases that represent different equivalence classes and boundary conditions. Unexpected inputs such as unsupported characters, incorrect formats,



excessive lengths, empty values, and unexpected combinations should be included where relevant to the requirements. For example, a numeric quantity field should reject invalid formats and values outside its permitted range while providing an appropriate error response. Testers should verify not only that invalid input is rejected but also that the application remains stable and does not produce unintended results. Proper validation testing improves robustness by preventing incorrect data from entering the system, exposes weaknesses in input handling, and reduces the likelihood of unexpected failures in production.

29. A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer: Frequent defect leakage into production usually indicates weaknesses in requirements of coverage, test design, risk assessment, regression testing, environment configuration, or release controls. The team should analyze escaped defects to identify their root causes and determine why existing tests did not detect them. High-risk and historically defect-prone modules should receive stronger coverage. Requirements should be traced to test cases, while boundary, negative, integration, usability, and regression scenarios should be included where appropriate. Defect trends can be reviewed to identify recurring gaps. Before release, exit criteria should require completion of critical tests and acceptable unresolved-defect levels. For example, a payment defect that escaped testing should lead to additional payment scenarios in the regression suite, not only a one-time fix. These practices reduce defect leakage, improve process quality, and increase confidence in production releases.

30. A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

Answer: The testing strategy should emphasize early involvement, risk-based planning, systematic test design, continuous feedback, and regression testing. Testers should review requirements and designs before coding, so ambiguity and missing conditions can be identified early. Requirements should be prioritized according to business and technical risk, and each important requirement should be linked to appropriate test cases. Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state-transition testing should be selected according to the problem. Unit, integration, system, and acceptance testing



should be performed at suitable stages supported by continuous regression after changes.

Defects should be documented with clear evidence and analyzed for root causes. Early defect detection reduces rework because problems are corrected closer to their source. A structured strategy therefore improves coverage, reduces cost and effort, and produces more reliable software.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand